Moving Ultrasound Radar

Group 31:
Intini Rocco
De Biasi Lorenzo
Gonzato Gabriele
Maria Leonardo

Embedded Software for the Internet of Things
Year 2025/2026

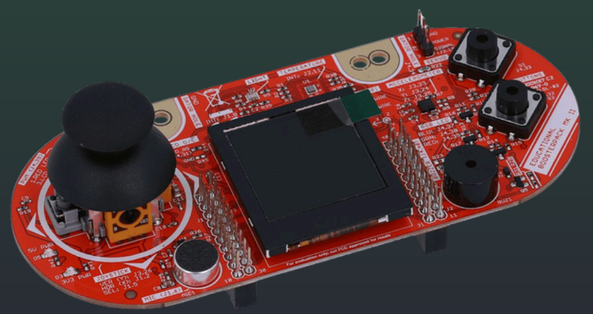
Teacher:
Proff. Yildirim

UNIVERSITÀ DI TRENTO

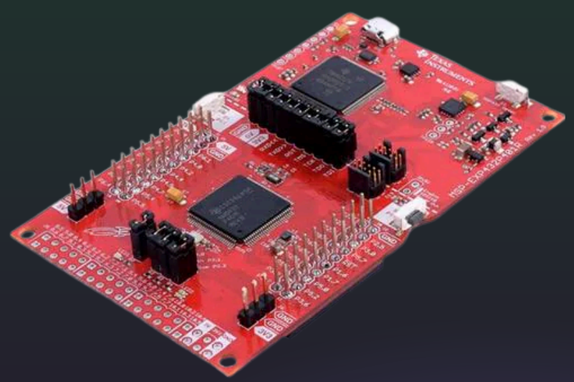
What is it?

A **rotating radar** that detects objects within a **4-meter range**, utilizing a servo motor to cover a **180-degree scanning arc**.
Real-time data is visualized through an **on-board display** and synced to a dedicated **webpage**, providing a **graphical sonar-style interface** for remote monitoring.

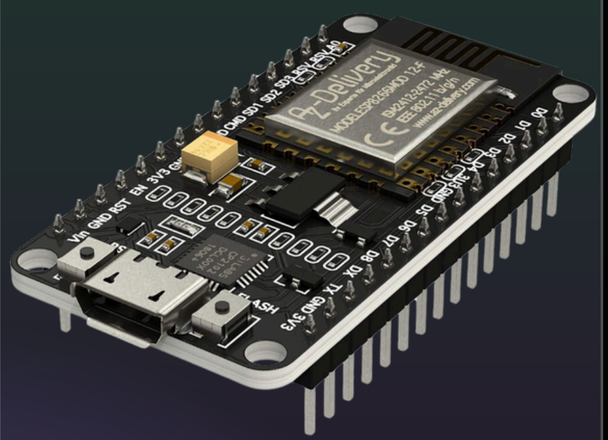
Components list



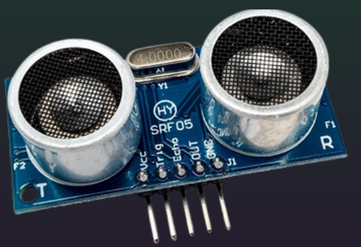
Boosterpack MKII



MSP432P401R



ESP-12E



Sensore HY-SRF05

Servo SG-90

Software logic

The Radar is implemented by finite state machine which follows repeatedly a cyclical path.

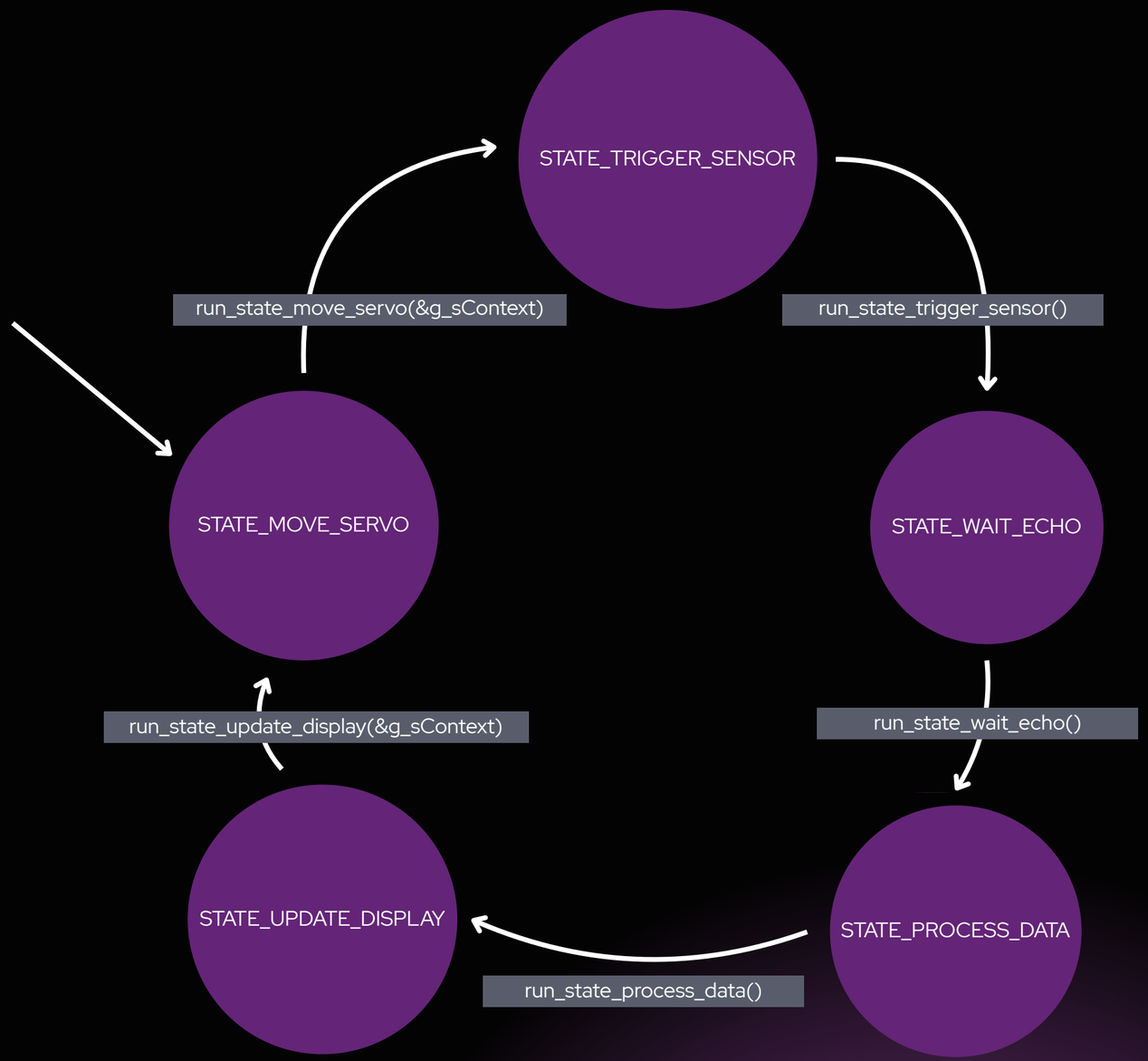
STATE_MOVE_SERVO is always the initial state, where we check the rotation limits to eventually change direction and set new angle

STATE_TRIGGER_SENSOR is responsible to begin a new echo capture

STATE_WAIT_ECHO: Manages asynchronous echo waiting by entering LPM0 for energy efficiency until an interrupt triggers data processing.

STATE_PROCESS_DATA compute the new angle to be sent at the Servo component

Finally **STATE_UPDATE_DISPLAY** calls Display's functions to show the data retrieved and start the new cycle



Key Code Snippet

```
void PORT3_IRQHandler(void){

    uint32_t status = MAP_GPIO_getEnabledInterruptStatus(GPIO_PORT_P3);
    MAP_GPIO_clearInterruptFlag(GPIO_PORT_P3, status);


    if(status & ECHO_PIN){


        if(MAP_GPIO_getInputPinValue(ECHO_PORT, ECHO_PIN)){

            t_start = TIMER_A1->R;


            MAP_GPIO_interruptEdgeSelect(ECHO_PORT, ECHO_PIN, GPIO_HIGH_TO_LOW_TRANSITION);

        } else{
            uint32_t t_end = TIMER_A1->R;


            t_diff = (t_end >= t_start) ? (t_end - t_start) : (0xFFFF - t_start + t_end);

            capture_done = true;


            MAP_Timer_A_disableCaptureCompareInterrupt(TIMER_A1_BASE, TIMER_A_CAPTURECOMPARE_REGISTER_0);

            MAP_GPIO_interruptEdgeSelect(ECHO_PORT, ECHO_PIN, GPIO_LOW_TO_HIGH_TRANSITION);


            MAP_Interrupt_disableSleepOnIsrExit();
        }
    }
}
```

Hardware test

01

file: test_display.c

Draws a static radar map and a simulated scanning line to verify SPI communication and the Graphics Library setup.

02

file: test_sensor.c

Triggers the ultrasonic sensor and prints the measured distance (in cm) to the console to verify timing and signal integrity.

03

file: test_servo.c

Validates PWM generation by moving the servo through three key positions (0°, 90°, and 180°) with a 2-second delay between steps.

04

file: test_uart_iot.c

Tests UART communication with the ESP-12E module by sending mock CSV data to the web dashboard, ensuring the cloud-link is active

Logic test

01

file: test_fsm_logic.c

There are implemented in this file all the states of the Finite State Machine that are used on the project, with all their transition functions. To ensure the logic works independently of the physical setup, hardware-dependent instructions were replaced with mock functions. These functions return predefined fake values or print status messages to the console to simulate real-world behavior.

Project Roles and Contributions

Intini Rocco

developed the IoT architecture and the HTML interface for radar visualization.

De Biasi Lorenzo

handled the servo-driven radar movement.

Gonzato Gabriele

was responsible for sensor configuration and calibration

Maria Leonardo

managed data visualization on the local display

Finally, the entire team collaborated on the main logic and the Finite State Machine (FSM) implementation.